

Trust Nothing: RTOS Security without Run-Time Software TCB (Additional Evaluation Data)

Eric Ackermann, Sven Bugiel
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
{eric.ackermann,bugiel}@cispa.de

A Detailed Evaluation Results

In order to facilitate transparency, we provide additional measurement values for our evaluation experiments. [Table 1](#) and [Table 4](#) list detailed values for our compute and scheduler benchmark results. [Table 3](#) contains detailed results from our IRQ benchmarks. In addition to Zephyr, Linux and Skadi on Bredi, we also include the upstream cva6 SoC as a reference. [Table 2](#) provides detailed results of our network benchmarks. Due to the upstream cva6 SoC using a different networking subsystem, this reference is excluded here.

Table 1: Full results from compute benchmark (Coremark) and memory benchmark (Stream) for Skadi, Zephyr and Linux. Also includes Zephyr on the upstream cva6 FPGA SoC. For Coremark, we provide the number of iterations, average duration and standard deviation from five runs as well as the standard metric, which is the median of the average numbers of iterations per second per run. For Stream, we provide results as reported.

Benchmark	Zephyr cva6 SoC	Linux Bredi	Zephyr Bredi	Skadi Bredi
Coremark Iterations	2000	1100	2000	2000
Median Coremark Iterations/sec	110.00	92.10	110.02	110.14
Coremark Duration avg / min / max / stddev/s	18.18 / 18.18 / 18.18 / 0	11.95 / 11.93 / 11.96 / 0.01	18.18 / 18.18 / 18.18 / 0	18.16 / 18.16 / 18.16 / 0.00
Stream Copy Best Rate/ MB/s	29.6	14.3	26.7	26.7
Stream Copy avg / min / max / stddev/s	5.41 / 5.40 / 5.50 / 0.03	11.23 / 11.22 / 11.23/-	6.04 / 6.00 / 6.10 / 0.05	6.00 / 6.00 / 6.00 / 0.00
Stream Scale Best Rate/ MB/s	25.4	12.6	24.2	24.1
Stream Scale avg / min / max / stddev/s	6.34 / 6.30 / 6.40 / 0.05	12.75 / 12.75 / 12.76/-	6.66 / 6.60 / 6.70 / 0.05	6.65 / 6.65 / 6.65 / 0.00
Stream Add Best Rate/ MB/s	27.6	13.7	26.7	26.6
Stream Add avg / min / max / stddev/s	8.73 / 8.70 / 8.80 / 0.05	17.53 / 17.52 / 17.60/-	9.03 / 9.00 / 9.10 / 0.05	9.02 / 9.02 / 9.02 / 0.00
Stream Triad Best Rate/ MB/s	27.6	12.9	25.3	25.3
Stream Triad avg / min / max / stddev/s	8.77 / 8.70 / 8.80 / 0.05	18.62 / 19.00 / 18.63/-	9.50 / 9.50 / 9.50 / 0	9.50 / 9.50 / 9.50 / 0.00

Table 2: Full results from networking benchmark for Skadi, Zephyr and Linux.

Benchmark	Linux	Zephyr	Skadi
Ping latency min / avg / max / stddev ms	1.77 / 1.92 / 2.62 / 0.15	0.83 / 0.88 / 1.08 / 0.02	2.46 / 2.56 / 3.96 / 0.08
iperf min / max / avg / stddev throughput TCP in Kbits/s	7580 / 14300 / 9031.17 / 1733.20	4380 / 4410 / 4396.67 / 6.81	1490 / 1500 / 1499.33 / 2.52
RX network stack processing durations ns min / max / avg / stddev	1212720 / 1721960 / 1346524.21 / 136558.72	516193 / 564497 / 534142.81 / 8120.03	2111305 / 2180633 / 2138493.63 / 13707.45
TX network stack processing durations ns min / max / avg / stddev	765200 / 1827240 / 1050532.63 / 203162.69	514855 / 644431 / 530697.96 / 13202.11	1491231 / 1559519 / 1516145.02 / 11670.62
MQTT-TLS Iteration duration ns min / max / avg / stddev	-	1993429040 / 1993582320 / 1993484845.47 / 30590.98	2049993880 / 2065165720 / 2057174000.84 / 5772516.87

Table 3: Results from real-time performance benchmarks for Skadi, Zephyr and Linux. Also includes Zephyr on the upstream cva6 FPGA SoC.

Benchmark	Zephyr cva6 SoC	Linux Bredi	Zephyr Bredi	Skadi Bredi
IRQ Latency independent min / max / avg / stddev ns:	4820 / 88140 / 12050.74 / 21226.22	1324020 / 9236700 / 3808793.68 / 3194695.59	4940 / 89960 / 24127.16 / 32149.03	65720 / 72840 / 68423.16 / 1529.02
IRQ Latency monotonic min period / min / max / avg / stddev ns:	68000 / 4740 / 62820 / 6061.68 / 7212.84	1312640 / 16432080 / 6866233.68 / 3989003.35 / 5691090.28	70000 / 4860 / 69740 / 8392.42 / 12120.04	136000 / 41300 / 114860 / 58826.32 / 18714.18

Table 4: Detailed results from scheduler macrobenchmark for Skadi and Zephyr. Also includes Zephyr on the upstream cva6 FPGA SoC. Names are as used in original Zephyr benchmark.

Macrobenchmark	Zephyr cva6 min / max / avg / std-dev ns	Zephyr Bredi min / max / avg / stddev ns	Skadi Bredi min / max / avg / std-dev ns
thread.yield.preemptive.ctx	4000 / 4960 / 4003.70 / 47.57	4040 / 4880 / 4067.06 / 64.08	13600 / 15840 / 13719.04 / 275.05
thread.yield.cooperative.ctx	4000 / 4960 / 4005.51 / 56.91	4040 / 4600 / 4061.95 / 36.63	13600 / 16560 / 13715.94 / 295.71
isr.resume.interrupted.thread.kernel - Return from ISR to interrupted thread	5080 / 7080 / 5934.75 / 124.00	5520 / 5840 / 5625.17 / 122.47	176800 / 198160 / 187828.02 / 3154.82
isr.resume.different.thread.kernel - Return from ISR to another thread	4800 / 5080 / 4839.92 / 60.18	5080 / 5400 / 5102.75 / 51.76	168520 / 188720 / 179111.04 / 3097.12
thread.create.kernel.from.kernel - Create thread	1727320 / 1805440 / 1765850.25 / 11137.18	2173200 / 2266240 / 2220408.44 / 13927.67	2594040 / 2692720 / 2642536.44 / 15995.25
thread.start.kernel.from.kernel - Start thread	8800 / 15640 / 12421.15 / 954.30	9240 / 15280 / 12168.32 / 900.14	87040 / 103040 / 95272.40 / 2539.70
thread.suspend.kernel.from.kernel - Suspend thread	6520 / 9520 / 8426.09 / 488.19	6000 / 9120 / 8254.19 / 420.37	55480 / 68040 / 61603.74 / 1968.75
thread.resume.kernel.from.kernel - Resume thread	4760 / 6720 / 5746.77 / 301.32	4840 / 6440 / 5699.66 / 259.41	52120 / 64800 / 57205.87 / 1813.72
thread.abort.kernel.from.kernel - Abort thread	6480 / 10480 / 9197.83 / 585.67	6520 / 10440 / 8873.57 / 576.07	1745440 / 1821880 / 1784033.93 / 13360.66
fifo.put.immediate.kernel - Add data to FIFO (no ctx switch)	3680 / 4280 / 3683.50 / 44.93	4160 / 4680 / 4189.51 / 44.12	32520 / 35320 / 32751.60 / 379.34
fifo.get.immediate.kernel - Get data from FIFO (no ctx switch)	2200 / 2240 / 2220.18 / 20.00	2280 / 2800 / 2280.68 / 17.23	26400 / 28000 / 26517.03 / 222.98
fifo.put.alloc.immediate.kernel - Allocate to add data to FIFO (no ctx switch)	11800 / 12880 / 11858.65 / 79.53	11760 / 12800 / 11814.75 / 84.59	32000 / 34440 / 32446.15 / 307.58
fifo.get.free.immediate.kernel - Free when getting data from FIFO (no ctx switch)	8480 / 9080 / 8511.24 / 60.18	8360 / 8880 / 8387.90 / 55.10	26280 / 29560 / 26362.45 / 217.27
fifo.get.blocking.k_to_k - Get data from FIFO (w/ ctx switch)	5600 / 6440 / 5655.96 / 80.33	5560 / 6200 / 5601.61 / 78.28	18400 / 21880 / 19562.41 / 476.69
fifo.put.wake+ctx.k_to_k - Add data to FIFO (w/ ctx switch)	8720 / 9320 / 8751.72 / 58.60	8720 / 9760 / 8747.90 / 62.04	23600 / 25640 / 23807.12 / 266.45
fifo.get.free.blocking.k_to_k - Free when getting data from FIFO (w/ ctx switch)	5720 / 6680 / 5831.20 / 129.82	5360 / 6000 / 5368.36 / 62.29	18800 / 21640 / 19073.69 / 457.81
fifo.put.alloc.wake+ctx.k_to_k - Allocate to add data to FIFO (w/ ctx switch)	8880 / 9480 / 8928.88 / 53.02	8840 / 9360 / 8858.37 / 59.92	23400 / 25720 / 23683.90 / 280.99
lifo.put.immediate.kernel - Add data to LIFO (no ctx switch)	3800 / 4400 / 3832.00 / 57.04	3800 / 4360 / 3827.90 / 48.25	31720 / 33880 / 31930.09 / 337.86
lifo.get.immediate.kernel - Get data from LIFO (no ctx switch)	2040 / 2160 / 2059.94 / 20.24	2120 / 2640 / 2121.05 / 23.29	26520 / 28240 / 26596.54 / 189.74
lifo.put.alloc.immediate.kernel - Allocate to add data to LIFO (no ctx switch)	11880 / 12760 / 11935.52 / 90.33	11760 / 12520 / 11800.40 / 70.76	31760 / 34000 / 31972.82 / 330.87
lifo.get.free.immediate.kernel - Free when getting data from LIFO (no ctx switch)	8440 / 9080 / 8497.65 / 52.86	7640 / 8240 / 7646.59 / 40.08	26240 / 28560 / 26335.72 / 211.59
lifo.get.blocking.k_to_k - Get data from LIFO (w/ ctx switch)	5400 / 6400 / 5440.36 / 109.40	5800 / 6720 / 5845.51 / 99.03	18680 / 21800 / 19296.08 / 495.55
lifo.put.wake+ctx.k_to_k - Add data to LIFO (w/ ctx switch)	8240 / 8840 / 8260.26 / 46.74	8240 / 8760 / 8267.70 / 48.32	23360 / 25120 / 23523.62 / 262.69
lifo.get.free.blocking.k_to_k - Free when getting data from LIFO (w/ ctx switch)	5080 / 5720 / 5084.98 / 48.46	5120 / 6280 / 5371.02 / 250.42	18280 / 20920 / 18530.25 / 421.76
lifo.put.alloc.wake+ctx.k_to_k - Allocate to add data to LIFO (w/ ctx switch)	8040 / 8640 / 8043.62 / 44.07	8480 / 9040 / 8485.47 / 36.26	22960 / 25360 / 23072.88 / 247.26

events.post.immediate.kernel - Post events (nothing wakes)	4320 / 5280 / 4373.23 / 62.51	4280 / 4600 / 4317.59 / 38.72	17720 / 18880 / 17798.15 / 181.44
events.set.immediate.kernel - Set events (nothing wakes)	3280 / 3880 / 3300.70 / 44.70	3360 / 3920 / 3363.98 / 44.35	17720 / 19440 / 17806.31 / 196.35
events.wait.immediate.kernel - Wait for any events (no ctx switch)	2480 / 2680 / 2497.45 / 20.63	2560 / 3040 / 2560.48 / 15.21	16560 / 17960 / 16655.12 / 188.94
events.wait_all.immediate.kernel - Wait for all events (no ctx switch)	2080 / 2680 / 2081.25 / 26.90	2120 / 2680 / 2140.54 / 26.31	16960 / 18240 / 17039.48 / 168.05
events.wait.blocking.k_to_k - Wait for any events (w/ ctx switch)	5400 / 6480 / 5682.37 / 187.66	5760 / 6520 / 5812.70 / 80.52	19960 / 23760 / 20954.17 / 664.17
events.set.wake+ctx.k_to_k - Set events (w/ ctx switch)	11480 / 12120 / 11548.78 / 77.56	11000 / 12120 / 11302.39 / 123.53	24120 / 27400 / 24982.95 / 500.72
events.wait_all.blocking.k_to_k - Wait for all events (w/ ctx switch)	5960 / 6800 / 5988.22 / 100.26	5960 / 7120 / 6057.61 / 185.03	19840 / 24120 / 20589.43 / 526.27
events.post.wake+ctx.k_to_k - Post events (w/ ctx switch)	11120 / 12160 / 11462.75 / 165.09	11080 / 11800 / 11110.87 / 87.48	25360 / 27480 / 25949.43 / 407.87
semaphore.give.immediate.kernel - Give a semaphore (no waiters)	2520 / 3120 / 2537.33 / 27.03	2600 / 2600 / 2600.00 / 0.00	17240 / 19400 / 17323.18 / 195.38
semaphore.take.immediate.kernel - Take a semaphore (no blocking)	1600 / 1640 / 1615.72 / 19.54	1680 / 1680 / 1680.00 / 0.00	16440 / 18400 / 16545.37 / 202.81
semaphore.take.blocking.k_to_k - Take a semaphore (context switch)	4560 / 5480 / 4594.25 / 98.46	4640 / 5280 / 4650.61 / 74.18	19480 / 23600 / 20730.89 / 655.34
semaphore.give.wake+ctx.k_to_k - Give a semaphore (context switch)	8200 / 8840 / 8253.87 / 59.42	7880 / 8400 / 7883.42 / 39.12	23400 / 27560 / 24482.81 / 479.94
condvar.wait.blocking.k_to_k - Wait for a condvar (context switch)	7960 / 9800 / 8300.42 / 248.81	7920 / 9720 / 7997.11 / 139.09	27800 / 30960 / 28254.87 / 557.20
condvar.signal.wake+ctx.k_to_k - Signal a condvar (context switch)	10280 / 11280 / 10334.79 / 83.94	10120 / 10880 / 10163.70 / 77.92	23680 / 26080 / 23806.55 / 260.22
stack.push.immediate.kernel - Add data to k_stack (no ctx switch)	2680 / 3240 / 2701.75 / 32.25	2760 / 3600 / 2761.61 / 31.18	17960 / 20960 / 18153.81 / 334.97
stack.pop.immediate.kernel - Get data from k_stack (no ctx switch)	2800 / 3160 / 2852.58 / 48.89	2840 / 3120 / 2861.03 / 54.69	20880 / 22640 / 21023.96 / 200.79
stack.pop.blocking.k_to_k - Get data from k_stack (w/ ctx switch)	5120 / 6200 / 5178.09 / 173.92	5240 / 5760 / 5242.89 / 35.52	22440 / 26760 / 23344.40 / 822.05
stack.push.wake+ctx.k_to_k - Add data to k_stack (w/ ctx switch)	8440 / 9040 / 8451.10 / 54.21	8200 / 8960 / 8452.30 / 53.83	23280 / 25480 / 23410.61 / 273.15
mutex.lock.immediate.recursive.kernel - Lock a mutex	1960 / 2640 / 1961.25 / 27.90	2000 / 2680 / 2021.67 / 36.56	16560 / 18360 / 16670.11 / 217.02
mutex.unlock.immediate.recursive.kernel - Unlock a mutex	1440 / 6680 / 1462.51 / 168.14	1520 / 6320 / 1525.87 / 153.83	15640 / 24840 / 15763.30 / 369.33
heap.malloc.immediate - Average time for heap malloc	7120 / 7120 / 7120.00 / 0.00	6800 / 7360 / 6805.89 / 57.15	5360 / 6120 / 5420.21 / 127.93
heap.free.immediate - Average time for heap free	7120 / 7120 / 7120.00 / 0.00	6800 / 7360 / 6805.89 / 57.15	5360 / 6120 / 5420.21 / 127.93